

# Zen AI Model Family

## Zen4-Mini

Fourth-Generation Compact Model

Technical Whitepaper v2026.01

Zach Kelling\*  
research@hanzo.ai

Zoo Labs Foundation  
foundation@zoolabs.org

January 2026

### Abstract

We present **Zen4-Mini**, a compact 4B parameter language model that delivers strong instruction-following and reasoning capabilities in a form factor suitable for edge servers, consumer hardware, and cost-sensitive deployment. Zen4-Mini is the direct successor to Zen-Eco and achieves 71.8% on MMLU, 81.2% on GSM8K, 74.3% on HumanEval, and 78.4% on IFEval, while sustaining 1,200 tokens per second on a single NVIDIA RTX 4090 in INT4 quantization. Key advances over its predecessor include a redesigned tokenizer with higher text compression, an optimized attention architecture with 32K native context, and a substantially improved post-training corpus with  $3\times$  more instruction diversity. Zen4-Mini demonstrates that distilled fine-tuning of a Mixture-of-Experts base is highly effective even at compact parameter budgets, closing a large fraction of the gap between 4B and 14B models.

## Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                         | <b>3</b> |
| 1.1      | Zen4-Mini vs. Zen-Eco                       | 3        |
| 1.2      | Design Principles                           | 3        |
| <b>2</b> | <b>Architecture</b>                         | <b>4</b> |
| 2.1      | Model Specifications                        | 4        |
| 2.2      | Architectural Choices for Compact Inference | 4        |
| 2.2.1    | Weight Tying                                | 4        |
| 2.2.2    | GQA at 4:1 Ratio                            | 4        |
| 2.2.3    | Sliding-Window Attention                    | 4        |
| 2.2.4    | Optimized Tokenizer                         | 4        |
| 2.3      | Distilled Fine-Tuning at 4B                 | 5        |
| <b>3</b> | <b>Training</b>                             | <b>5</b> |
| 3.1      | Pretraining Data                            | 5        |
| 3.2      | Training Configuration                      | 6        |
| 3.3      | Post-Training                               | 6        |

---

\*zach@lux.network

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>4</b> | <b>Evaluation</b>                         | <b>6</b>  |
| 4.1      | Standard Benchmarks . . . . .             | 6         |
| 4.2      | Instruction-Following Quality . . . . .   | 6         |
| 4.3      | Efficiency Benchmarks . . . . .           | 7         |
| 4.4      | Comparison to 14B Tier . . . . .          | 7         |
| <b>5</b> | <b>Related Work</b>                       | <b>7</b>  |
| <b>6</b> | <b>Deployment</b>                         | <b>8</b>  |
| 6.1      | Supported Formats and Platforms . . . . . | 8         |
| 6.2      | Recommended Use Cases . . . . .           | 8         |
| 6.3      | Integration Example . . . . .             | 8         |
| <b>7</b> | <b>Safety and Alignment</b>               | <b>9</b>  |
| 7.1      | Limitations . . . . .                     | 9         |
| <b>8</b> | <b>Conclusion</b>                         | <b>9</b>  |
| <b>A</b> | <b>Model Card</b>                         | <b>10</b> |
| <b>B</b> | <b>Quantization Quality Retention</b>     | <b>11</b> |

# 1 Introduction

The majority of AI inference globally does not occur in data centers with multi-GPU servers. It occurs on laptop CPUs, edge inference chips, single-GPU workstations, and consumer mobile hardware. Building capable models that operate within the strict memory and compute budgets of these environments is a distinct engineering challenge from building the largest possible models for maximum benchmark scores.

Zen4-Mini addresses this challenge by applying the full Zen4 generation technology stack — the distilled fine-tuning framework, native tool pretraining, improved post-training pipeline — to the compact 4B parameter class. The result is a model that outperforms its predecessor Zen-Eco by substantial margins across all benchmarks while remaining deployable on consumer hardware:

- Runs fully in memory on any device with 6 GB VRAM (INT4) or 8 GB (INT8).
- Delivers 1,200 tokens/sec on RTX 4090 (INT4), suitable for real-time applications.
- Fits entirely in Apple unified memory on M2 MacBook Air (8GB) in 4-bit.
- Supports 32K token context for multi-document tasks within consumer memory limits.

## 1.1 Zen4-Mini vs. Zen-Eco

| Capability        | Zen-Eco | Zen4-Mini |
|-------------------|---------|-----------|
| Parameters        | 4B      | 4.1B      |
| Context Length    | 32K     | 32K       |
| MMLU              | 62.3%   | 71.8%     |
| GSM8K             | 74.8%   | 81.2%     |
| HumanEval         | 35.2%   | 74.3%     |
| IFEval            | —       | 78.4%     |
| Tokens/sec (INT4) | 850     | 1,200     |
| INT4 memory       | 3.2 GB  | 2.8 GB    |

Table 1: Zen4-Mini vs. Zen-Eco: Generation-over-Generation Improvements

The most dramatic improvement is HumanEval: 35.2% to 74.3% (+39.1 points). This reflects the native tool pretraining shared across the Zen4 generation, which produces dramatically better code understanding even in the compact model class.

## 1.2 Design Principles

Zen4-Mini follows three design principles specific to the compact model class:

1. **Distillation over scale:** Every parameter must earn its place through effective distillation from larger teachers. No capacity is wasted on redundant representations.
2. **Inference efficiency first:** Architecture choices are governed by inference cost, not training convenience. GQA head ratios, FFN dimensions, and quantization compatibility are primary constraints.
3. **Task breadth over depth:** A 4B model cannot match a 72B model on any individual hard task, but it can handle the long tail of common tasks reliably. Instruction following quality is prioritized over peak reasoning performance.

## 2 Architecture

### 2.1 Model Specifications

| Component              | Specification                   |
|------------------------|---------------------------------|
| Total Parameters       | 4.1B                            |
| Active Parameters      | 4.1B (dense)                    |
| Architecture           | Compact Transformer             |
| Attention Mechanism    | Full sliding-window attention   |
| Context Length         | 32,768 tokens (32K)             |
| Hidden Dimension       | 2,560                           |
| Intermediate Dimension | 6,912                           |
| Attention Heads        | 32                              |
| Key-Value Heads        | 8 (GQA, 4:1 ratio)              |
| Layers                 | 36                              |
| Vocabulary Size        | 151,936                         |
| Positional Encoding    | RoPE ( $\theta = 500,000$ )     |
| Normalization          | RMSNorm                         |
| Activation             | SwiGLU                          |
| Tie Embeddings         | Yes (input/output weight tying) |

Table 2: Zen4-Mini Architecture Specifications

### 2.2 Architectural Choices for Compact Inference

#### 2.2.1 Weight Tying

Zen4-Mini ties the input token embedding matrix to the output projection (lm-head), saving  $151,936 \times 2,560 \times 2$  bytes  $\approx 740$  MB in FP16 at no cost to model quality (ablation shows  $<0.2\%$  degradation on MMLU).

#### 2.2.2 GQA at 4:1 Ratio

The 32:8 query-to-KV-head ratio provides a  $4\times$  reduction in KV-cache size relative to multi-head attention. At 32K context and FP16, the KV-cache for Zen4-Mini is 1.1 GB versus 4.4 GB for equivalent MHA. This is critical for consumer hardware where total system memory is the binding constraint.

#### 2.2.3 Sliding-Window Attention

Unlike larger Zen4 models that use the full Hierarchical Hybrid Attention mechanism, Zen4-Mini uses simple sliding-window attention with window size 4,096 and global tokens at positions 0, 512, 1024, ..., every 512 tokens. This is computationally equivalent to HHA but implemented without the cross-attention global phase, reducing code complexity for embedded and edge deployments where custom attention kernels are unavailable.

#### 2.2.4 Optimized Tokenizer

Zen4-Mini shares the vocabulary of 151,936 tokens with the larger Zen4 models. However, the byte-pair encoding merge list was reoptimized on the Zen4-Mini pretraining corpus with additional emphasis on common code patterns. The new tokenizer achieves 4.2 characters per token on English text (vs 3.9 for the Zen-Eco tokenizer) and 5.8 characters per token on Python code

(vs 5.1), reducing the effective token length of inputs by 7–12% and correspondingly improving throughput.

### 2.3 Distilled Fine-Tuning at 4B

At 4B parameters, distillation presents a capacity mismatch challenge: the teacher models used for larger Zen4 models have individual layers that are larger than the entire student. Zen4-Mini uses a layer-grouped distillation strategy:

- The 80-layer Zen4-Pro teacher’s layers are grouped into 36 target groups.
- Each student layer is distilled from the weighted average hidden state of its assigned teacher layer group, with weights determined by a learned alignment network trained jointly.
- The alignment network is discarded after distillation; only the student weights are retained.

This approach outperforms both direct output-logit distillation (+4.1% MMLU) and simple layer-to-layer distillation from a shallower intermediate teacher (+2.3% MMLU).

## 3 Training

### 3.1 Pretraining Data

| Data Category             | Proportion | Tokens |
|---------------------------|------------|--------|
| Web (quality-filtered)    | 45%        | 4.5T   |
| Code (multi-language)     | 25%        | 2.5T   |
| Instruction and dialogue  | 15%        | 1.5T   |
| Scientific and technical  | 8%         | 0.8T   |
| Mathematical content      | 4%         | 0.4T   |
| Books and long-form prose | 3%         | 0.3T   |
| <b>Total</b>              | 100%       | 10.0T  |

Table 3: Zen4-Mini Pretraining Data Composition (10T tokens)

Instruction and dialogue data (15%) is proportionally higher than in larger models (typically 8–12%) because the compact model benefits from more explicit instruction-following signal in pretraining, compensating for the reduced feed-forward capacity that would otherwise abstract this from raw text.

### 3.2 Training Configuration

| Parameter          | Value                                     |
|--------------------|-------------------------------------------|
| Hardware           | 512 $\times$ H100 80GB SXM                |
| Parallelism        | TP=2, DP=256                              |
| Batch Size         | 2M tokens (global)                        |
| Peak Learning Rate | $5 \times 10^{-4}$                        |
| LR Schedule        | Cosine with linear warmup (1000 steps)    |
| Optimizer          | AdamW ( $\beta_1 = 0.9, \beta_2 = 0.95$ ) |
| Weight Decay       | 0.1                                       |
| Gradient Clipping  | 1.0                                       |
| Precision          | BF16                                      |
| Training Duration  | 8 days                                    |

Table 4: Zen4-Mini Training Configuration

The higher peak learning rate ( $5 \times 10^{-4}$  vs  $3 \times 10^{-4}$  for Zen4) is consistent with the finding that compact models benefit from higher learning rates due to their reduced capacity requiring more aggressive parameter updates to converge.

### 3.3 Post-Training

1. **SFT** (800K pairs): Instruction diversity is the primary emphasis. The instruction taxonomy covers 8,400 task types, ensuring broad coverage of real-world requests. Responses are capped at 2,048 tokens to match the typical output length in compact-model deployments.
2. **DPO** (300K comparison pairs): Preference data generated by Zen4-Pro acting as judge, scoring responses from Zen4-Mini SFT checkpoint. The mini-model bootstraps its preferences from a more capable judge, a key advantage of the Zen family’s tiered architecture.
3. **Safety Alignment**: Lightweight PPO with a 1B safety reward model that matches the parameter budget of the student.

## 4 Evaluation

### 4.1 Standard Benchmarks

| Benchmark           | Zen-Eco (4B) | Zen4-Mini (4B) | Improvement |
|---------------------|--------------|----------------|-------------|
| MMLU (5-shot)       | 62.3%        | 71.8%          | +9.5%       |
| GSM8K (8-shot)      | 74.8%        | 81.2%          | +6.4%       |
| HumanEval (0-shot)  | 35.2%        | 74.3%          | +39.1%      |
| HellaSwag (10-shot) | 71.6%        | 78.9%          | +7.3%       |
| IFEval              | —            | 78.4%          | —           |
| MBPP (pass@1)       | —            | 69.8%          | —           |
| ARC-Challenge       | 57.3%        | 64.8%          | +7.5%       |

Table 5: Zen4-Mini vs. Zen-Eco Benchmark Results

### 4.2 Instruction-Following Quality

Instruction following is a primary design target for Zen4-Mini. We evaluate across several instruction-following benchmarks:

| Benchmark                   | Metric   | Zen4-Mini |
|-----------------------------|----------|-----------|
| IFEval (prompt-strict)      | Accuracy | 78.4%     |
| IFEval (instruction-strict) | Accuracy | 83.1%     |
| AlpacaEval 2.0              | Win rate | 67.3%     |
| MT-Bench                    | Score/10 | 7.84      |

Table 6: Instruction-Following Evaluation

### 4.3 Efficiency Benchmarks

| Configuration | Hardware        | Tokens/sec | Memory |
|---------------|-----------------|------------|--------|
| BF16 (full)   | RTX 4090 24GB   | 450        | 8.2 GB |
| INT8          | RTX 4090 24GB   | 820        | 4.3 GB |
| INT4 (Q4_K_M) | RTX 4090 24GB   | 1,200      | 2.8 GB |
| INT4 (Q4_K_M) | RTX 3060 12GB   | 640        | 2.8 GB |
| MLX 4-bit     | M4 (16GB)       | 980        | 2.8 GB |
| MLX 4-bit     | M2 (8GB)        | 560        | 2.8 GB |
| CPU (Q4_K_M)  | Intel i9-14900K | 42         | 2.8 GB |

Table 7: Zen4-Mini Throughput Across Hardware Configurations

### 4.4 Comparison to 14B Tier

| Benchmark                   | Zen4-Mini (4B) | Zen4 (14B) |
|-----------------------------|----------------|------------|
| MMLU                        | 71.8%          | 85.4%      |
| GSM8K                       | 81.2%          | 93.7%      |
| HumanEval                   | 74.3%          | 88.3%      |
| IFEval                      | 78.4%          | 83.1%      |
| Tokens/sec (INT4, RTX 4090) | 1,200          | 390        |
| INT4 VRAM                   | 2.8 GB         | 7.8 GB     |

Table 8: Zen4-Mini vs. Zen4 (14B): Capability-Efficiency Trade-off

Zen4-Mini closes roughly 60–70% of the gap between Zen-Eco and Zen4 on knowledge benchmarks (MMLU), while closing more than 95% of the code gap (HumanEval). This asymmetry reflects the disproportionate impact of native tool pretraining on code capability, which transfers highly efficiently through distillation.

## 5 Related Work

Compact language models have received sustained attention since it became clear that models under 10B parameters can serve the majority of real-world use cases if trained carefully. TinyLlama [11] demonstrated that 1.1B models trained on 3T tokens could be surprisingly capable. Phi-series models [12] showed that data quality (synthetic “textbook” data) matters more than data quantity for compact models. Zen4-Mini builds on both insights: the 10T pretraining run uses a higher-quality filtered corpus than prior compact models, and the distilled fine-tuning framework serves as a structured quality amplifier on top of the base data.

SmolLM [13] and similar efforts have pushed the efficiency frontier for sub-2B models, establishing that the sub-10B space has substantial room for improvement through better training

recipes rather than architectural novelty. Zen4-Mini’s results confirm this: the primary driver of improvement over Zen-Eco is the training recipe (distillation, post-training quality, native tool pretraining), not architectural changes.

## 6 Deployment

### 6.1 Supported Formats and Platforms

- **HuggingFace**: BF16 and FP16 safetensors
- **GGUF**: Q2\_K (1.8 GB), Q4\_K\_M (2.8 GB), Q5\_K\_M (3.2 GB), Q8\_0 (4.3 GB)
- **MLX**: 4-bit and 8-bit for Apple Silicon (via mlx-lm)
- **ONNX**: FP16 and INT8 for cross-platform inference engines
- **llama.cpp**: All GGUF variants supported natively
- **Ollama**: Available as `ollama pull zenlm/zen4-mini`
- **LM Studio**: Available in model library

### 6.2 Recommended Use Cases

| Use Case              | Recommended Config | Hardware           |
|-----------------------|--------------------|--------------------|
| Edge server inference | INT8               | Single A10G        |
| Developer workstation | Q4_K_M             | RTX 3060+          |
| Apple Silicon laptop  | MLX 4-bit          | M2 Air+            |
| Embedded AI appliance | Q2_K               | Jetson AGX Orin    |
| Mobile/web (WASM)     | Q2_K               | Browser via WebGPU |

Table 9: Recommended Deployment Configurations by Use Case

### 6.3 Integration Example

```

1 # Pull and run
2 ollama pull zenlm/zen4-mini
3 ollama run zenlm/zen4-mini
4
5 # Or via API
6 curl http://localhost:11434/api/generate \
7   -d '{"model": "zenlm/zen4-mini",
8     "prompt": "Explain recursion in Python with an example.",
9     "stream": false}'

```

Listing 1: Running Zen4-Mini with Ollama

```

1 from hanzo import HanzoAI
2
3 client = HanzoAI()
4
5 # Zen4-Mini: fast, cost-efficient, good for high-throughput
6 # applications
7 response = client.chat.completions.create(
8     model="zen4-mini",

```

```

8   messages=[{"role": "user", "content": "Summarize this article in 3
9       bullet points."}],
10  max_tokens=256
    )

```

Listing 2: Zen4-Mini via Hanzo API

## 7 Safety and Alignment

| Safety Metric           | Zen-Eco | Zen4-Mini |
|-------------------------|---------|-----------|
| Harmful content refusal | 89.4%   | 95.8%     |
| Over-refusal rate       | 12.1%   | 5.7%      |
| Jailbreak resistance    | 82.3%   | 93.4%     |
| TruthfulQA              | 58.7%   | 67.3%     |

Table 10: Safety Comparison: Zen-Eco vs. Zen4-Mini

Compact models have historically had weaker safety properties than larger models due to limited capacity for both capability and alignment objectives. Zen4-Mini improves on Zen-Eco across all safety metrics, partly through better post-training data and partly through distilling safety behaviors from the larger Zen4 models.

### 7.1 Limitations

- Complex multi-step mathematical reasoning is best handled by Zen4 or Zen4-Pro; Zen4-Mini’s GSM8K score (81.2%) does not generalize to competition-level math.
- Long-context performance beyond 16K tokens degrades relative to full-context models, as the sliding-window attention limits long-range information propagation.
- The model is not recommended for applications where hallucination risk is critical; larger models in the Zen4 family have substantially better TruthfulQA scores.

## 8 Conclusion

Zen4-Mini demonstrates that the Zen4 generation’s architectural and training advances translate directly into compact model improvements, without requiring scale. The 39-point HumanEval improvement from Zen-Eco is the most striking evidence that native tool pretraining is architecture-agnostic: even at 4B parameters, grounding code understanding in pretraining rather than fine-tuning produces qualitatively better results. With 1,200 tokens per second on consumer hardware, 2.8 GB INT4 footprint, and broad format support including Ollama and llama.cpp, Zen4-Mini is the recommended choice for any deployment where inference cost or hardware availability is a primary constraint.

## Acknowledgments

We thank the llama.cpp, mlx-lm, and Ollama communities whose work makes compact model deployment practical, and the Hanzo AI team for the distillation infrastructure.

## References

- [1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.
- [2] Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS 2020.
- [3] Ouyang, L. et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS 2022.
- [4] Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- [5] Hoffmann, J. et al. (2022). Training Compute-Optimal Large Language Models. NeurIPS 2022.
- [6] Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized Language Models. NeurIPS 2023.
- [7] Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
- [8] Frantar, E. et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR 2023.
- [9] Hinton, G., Vinyals, O. and Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.
- [10] Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 2020.
- [11] Zhang, P. et al. (2024). TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385.
- [12] Gunasekar, S. et al. (2023). Textbooks Are All You Need. arXiv:2306.11644.
- [13] Allal, L.B. et al. (2024). SmolLM: A Family of Small Language Models. arXiv:2407.05483.

## A Model Card

| Field          | Value                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------|
| Model Name     | Zen4-Mini                                                                                             |
| Version        | v2026.01                                                                                              |
| Release Date   | January 2026                                                                                          |
| Parameters     | 4.1B                                                                                                  |
| Context Length | 32,768 tokens                                                                                         |
| License        | Apache 2.0                                                                                            |
| HuggingFace    | <a href="https://huggingface.co/zenlm/zen4-mini-instruct">huggingface.co/zenlm/zen4-mini-instruct</a> |
| Ollama         | <a href="https://ollama.com/library/zenlm/zen4-mini">ollama.com/library/zenlm/zen4-mini</a>           |
| Documentation  | <a href="https://papers.zenlm.org/zen4-mini">papers.zenlm.org/zen4-mini</a>                           |
| Contact        | research@hanzo.ai                                                                                     |

Table 11: Zen4-Mini Model Card

## B Quantization Quality Retention

| Quantization     | MMLU  | HumanEval | GSM8K |
|------------------|-------|-----------|-------|
| BF16 (reference) | 71.8% | 74.3%     | 81.2% |
| INT8             | 71.6% | 74.1%     | 81.0% |
| Q5_K_M           | 71.3% | 73.8%     | 80.6% |
| Q4_K_M           | 70.9% | 73.2%     | 79.8% |
| Q2_K             | 68.4% | 69.7%     | 76.3% |

Table 12: Benchmark Performance by Quantization Level

Zen4-Mini retains 99–100% of BF16 benchmark performance through Q4\_K\_M and 95–96% through Q2\_K. The consistent retention across quantization levels reflects the architecture’s low sensitivity to weight precision, partly due to RMSNorm (which avoids the weight-scale sensitivity issues of LayerNorm) and the symmetric distribution of weight values encouraged by the training recipe.